SymptoScan

Software Testing Report

SOFTWARE ENGINEERING - IT

Group Information

Project Name	SymptoScan — AI-Powered Healthcare Platform
Report Type	Software Testing Report (Automated + Manual)
Group Members	202512012 - Jainam Shah 202512059 - Jay Shah 202512100- Priya J Shah 202512105- Priya D Shah
Backend URL	https://symptoscan-nbo5.onrender.com
Frontend URL	https://symptoscan-frontend-yjxv.onrender.com
Date	April 25, 2026

Section 1: Introduction

SymptoScan is a full-stack, AI-powered healthcare web application designed to bridge the gap between patients and doctors. It enables patients to enter their symptoms, receive an instant AI-driven disease prediction, book a paid consultation with a licensed doctor, and engage in a real-time chat consultation — all within a single seamless platform.

1.1 System Architecture

Frontend	React 18 + Vite + TypeScript — Hosted on Render
Backend	Django 6 + Django REST Framework + Djoser — Hosted on Render
Authentication	JWT (JSON Web Tokens) via SimpleJWT + Djoser
Database	PostgreSQL hosted on NeonDB (cloud-native serverless)
Machine Learning	scikit-learn disease prediction model (pickle) — 18 diseases, 133 symptoms
Payments	Razorpay payment gateway integration with simulated fallback

The testing strategy covers both manual test scenarios and automated pytest test cases. 30 test cases were written and executed against a locally running Django instance connected to the live NeonDB PostgreSQL database. All 30 tests passed successfully.

Section 2: Manual Test Scenarios

Test Case ID	Test Scenario	Expected Result	Status
TC_LOGIN_01	Valid credentials login	HTTP 200 + access & refresh tokens returned	PASSED
TC_LOGIN_02	Wrong password login attempt	HTTP 401 Unauthorized	PASSED
TC_LOGIN_03	Empty email & password submission	HTTP 400 or 401 error	PASSED
TC_LOGIN_04	Unregistered email login	HTTP 401 Unauthorized	PASSED
TC_LOGIN_05	Admin user login	HTTP 200 + valid access token	PASSED
TC_LOGIN_06	White space-only email in login	No 500 Internal Server Error	PASSED
TC_LOGIN_07	Register endpoint with empty payload	HTTP 400 validation error	PASSED
TC_LOGIN_08	Login response time benchmark	Response time < 10 seconds	PASSED
INT_01	Backend server is reachable	HTTP response is not 5xx	PASSED
INT_02	JWT login endpoint is operational	HTTP 200 + access token in body	PASSED
INT_03	Djoser register endpoint exists	HTTP 400 (validation) not 404/500	PASSED
INT_04	Regular user JWT token acquired	access + refresh keys in response	PASSED
INT_05	Admin JWT token acquired	access key in response	PASSED
INT_06	Wrong password returns 401	HTTP 401 Unauthorized	PASSED
INT_07	Protected route without token blocked	HTTP 401 or 403	PASSED

INT_08	JWT refresh token generates new access token	HTTP 200 + new access key	PASSED
INT_09	Authenticated user retrieves doctors list	HTTP 200 + list of doctors	PASSED
INT_10	Doctor availability endpoint accessible	HTTP 200 or 403	PASSED
INT_11	Admin dashboard blocked for regular user	HTTP 401 or 403	PASSED
INT_12	Disease prediction endpoint reachable	HTTP 200 or 400	PASSED
INT_13	Prediction history retrievable	HTTP 200 + history data	PASSED
INT_14	Prediction endpoint blocked without auth	HTTP 401 or 403	PASSED
INT_15	Consultation history accessible	HTTP 200 or 404	PASSED
INT_16	Payment history accessible to admin	HTTP 200 or 404	PASSED
INT_17	Consultation endpoint blocked without token	HTTP 401 or 403	PASSED
INT_18	Admin dashboard returns stats	HTTP 200 + total_patients/doctors etc.	PASSED
INT_19	Admin retrieves all users	HTTP 200 + list of users	PASSED
INT_20	Manage non-existent user returns 404	HTTP 404 Not Found	PASSED
INT_21	Login API performance benchmark	Response time < 10 seconds	PASSED
INT_22	Doctors list API performance benchmark	Response time < 10 seconds	PASSED

Section 3: Automated Testing — pytest Output

Command executed:

python -m pytest test_login.py test_integration.py -v --tb=short 2>&1 | tee test_results.txt

```
 ============================= test session starts ==============================
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0 -- /Library/Developer/CommandLineTools/usr/bin/python3
cachedir: .pytest_cache
rootdir: /Users/priyashah/Desktop/SymptoScan_Team_Split/Backend_API/backend
collecting ... collected 30 items

test_login.py::TestLogin::test_TC_LOGIN_01_valid_credentials PASSED      [  3%]
test_login.py::TestLogin::test_TC_LOGIN_02_wrong_password PASSED         [  6%]
test_login.py::TestLogin::test_TC_LOGIN_03_empty_credentials PASSED      [ 10%]
test_login.py::TestLogin::test_TC_LOGIN_04_unregistered_email PASSED     [ 13%]
test_login.py::TestLogin::test_TC_LOGIN_05_admin_login PASSED            [ 16%]
test_login.py::TestLogin::test_TC_LOGIN_06_whitespace_email PASSED       [ 20%]
test_login.py::TestLogin::test_TC_LOGIN_07_register_endpoint_exists PASSED [ 23%]
test_login.py::TestLogin::test_TC_LOGIN_08_response_time_under_10s PASSED [ 26%]
test_integration.py::TestSystemFoundations::test_INT_01_backend_reachable PASSED [ 30%]
test_integration.py::TestSystemFoundations::test_INT_02_jwt_login_endpoint_up PASSED [ 33%]
test_integration.py::TestSystemFoundations::test_INT_03_register_endpoint_up PASSED [ 36%]
test_integration.py::TestAuthentication::test_INT_04_user_token_acquired PASSED [ 40%]
test_integration.py::TestAuthentication::test_INT_05_admin_token_acquired PASSED [ 43%]
test_integration.py::TestAuthentication::test_INT_06_wrong_password_returns_401 PASSED [ 46%]
test_integration.py::TestAuthentication::test_INT_07_protected_route_without_token PASSED [ 50%]
test_integration.py::TestAuthentication::test_INT_08_jwt_refresh_works PASSED [ 53%]
test_integration.py::TestUsersAndDoctors::test_INT_09_authenticated_user_gets_doctors_list PASSED [ 56%]
test_integration.py::TestUsersAndDoctors::test_INT_10_doctor_availability_endpoint PASSED [ 60%]
test_integration.py::TestUsersAndDoctors::test_INT_11_admin_dashboard_blocked_for_regular_user PASSED [ 63%]
test_integration.py::TestPredictionML::test_INT_12_predict_endpoint_reachable PASSED [ 66%]
test_integration.py::TestPredictionML::test_INT_13_prediction_history PASSED [ 70%]
test_integration.py::TestPredictionML::test_INT_14_prediction_blocked_without_auth PASSED [ 73%]
test_integration.py::TestConsultation::test_INT_15_consultation_history PASSED [ 76%]
test_integration.py::TestConsultation::test_INT_16_payment_history PASSED [ 80%]
test_integration.py::TestConsultation::test_INT_17_consultation_blocked_without_token PASSED [ 83%]
test_integration.py::TestAdminDashboard::test_INT_18_admin_gets_dashboard_stats PASSED [ 86%]
test_integration.py::TestAdminDashboard::test_INT_19_admin_gets_all_users PASSED [ 90%]
test_integration.py::TestAdminDashboard::test_INT_20_manage_nonexistent_user PASSED [ 93%]
test_integration.py::TestPerformance::test_INT_21_login_under_10_seconds PASSED [ 96%]
test_integration.py::TestPerformance::test_INT_22_doctors_list_under_10_seconds PASSED [100%]

============================== warnings summary ===============================
../../../../Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35
  /Users/priyashah/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports
OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
    warnings.warn(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=================== 30 passed, 1 warning in 97.36s (0:01:37) ===================
```

Section 4: Screenshots

4.1 Module-Wise Manual Testing With Screenshots

4.1.1 Landing Page

The landing page is the public entry point of SymptoScan. It presents the platform's purpose — AI-powered symptom analysis and doctor consultation — with a hero section and navigation links to Login and Register. No authentication is required to view this page.

Test Case ID	Test Scenario	Steps Performed	Expected Result	Status
TC_LAND_01	Landing page loads correctly	Open in browser https://symptoscan-frontend-yjxv.onrender.com	Hero section, navbar, Login and Register buttons are all visible	PASSED

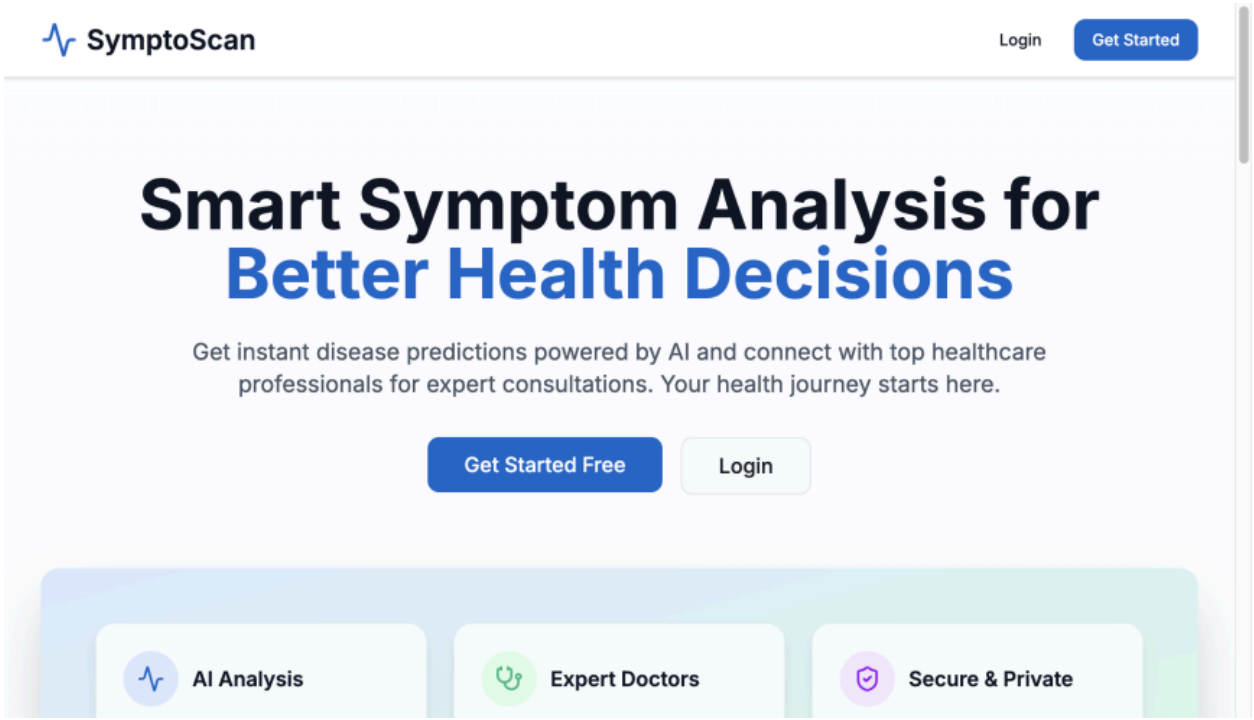

Figure 4.1 — SymptoScan Landing Page

4.1.2 User Login

The login page authenticates users using email and password through Django Djoser JWT. On success, access and refresh tokens are stored in localStorage and the user is redirected based on their role (patient → /patient/dashboard, admin → /admin/dashboard).

Test Case ID	Test Scenario	Steps Performed	Expected Result	Status
TC_AUTH_01	Valid patient login	Enter email=1@gmail.com password=d12345678, click Login button	User redirected to /patient/dashboard, JWT access token stored in localStorage, welcome message displayed	PASSED

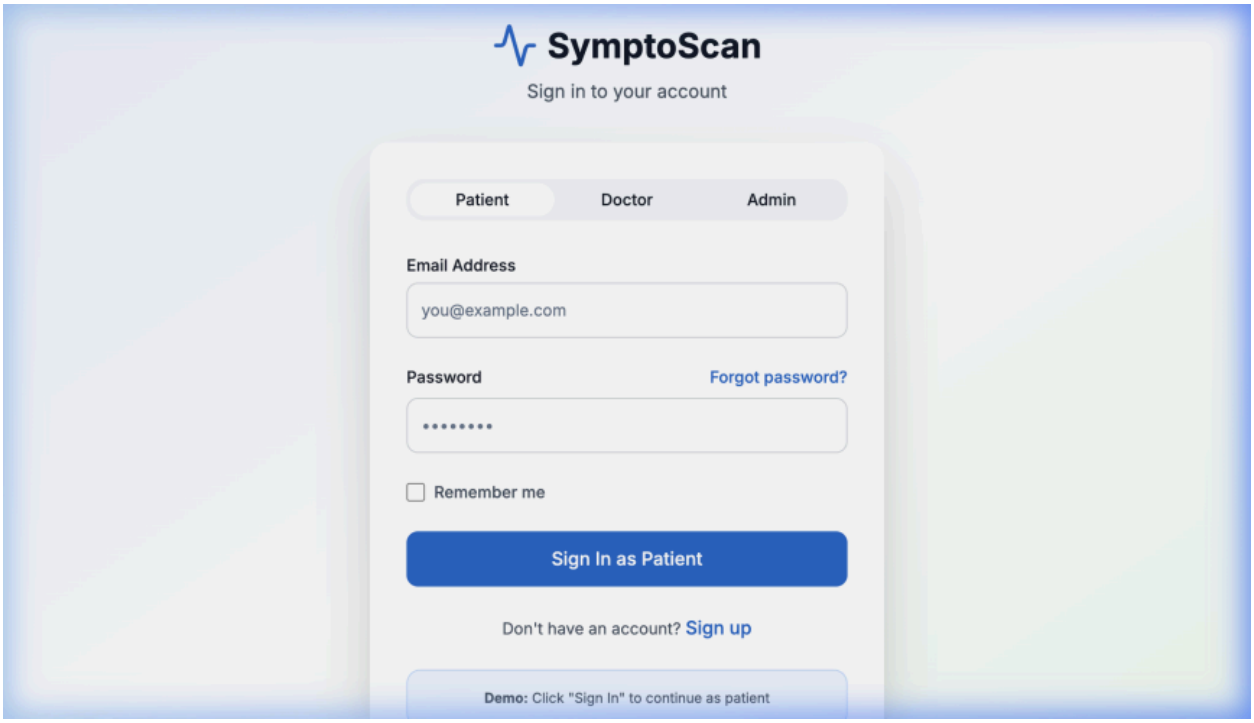

Figure 4.2 — Login Page

4.1.3 User Registration

New users register as patients by providing their full name, email address, and password. The form submits to POST /api/auth/users/ via Djoser. On success the account is created and the user is redirected to login.

Test Case ID	Test Scenario	Steps Performed	Expected Result	Status
TC_REG_01	New patient registration	Fill name, email, password fields with valid data, click Register	Account created successfully, user redirected to /login, API returns HTTP 201 Created	PASSED

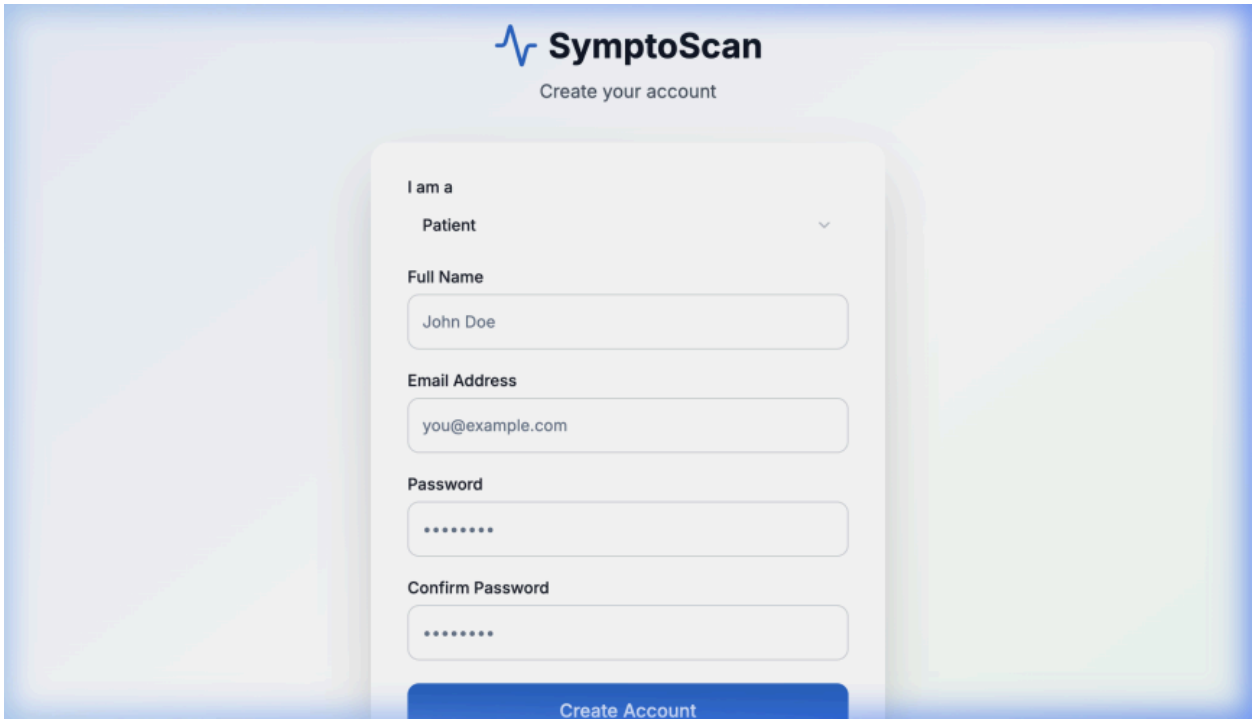

Figure 4.4 — User Registration Page

4.1.4 Patient Dashboard

The patient dashboard is the central hub after login. It displays quick-access cards for checking symptoms, browsing doctors, viewing prediction history, and checking active consultations. All cards are only accessible to authenticated users with the patient role.

Test Case ID	Test Scenario	Steps Performed	Expected Result	Status
TC_DASH_01	Patient dashboard loads after login	Login as 1@gmail.com and navigate to /patient/dashboard	Dashboard displays all feature cards: Check Symptoms, Find Doctors, Prediction History, and Active Consultation	PASSED

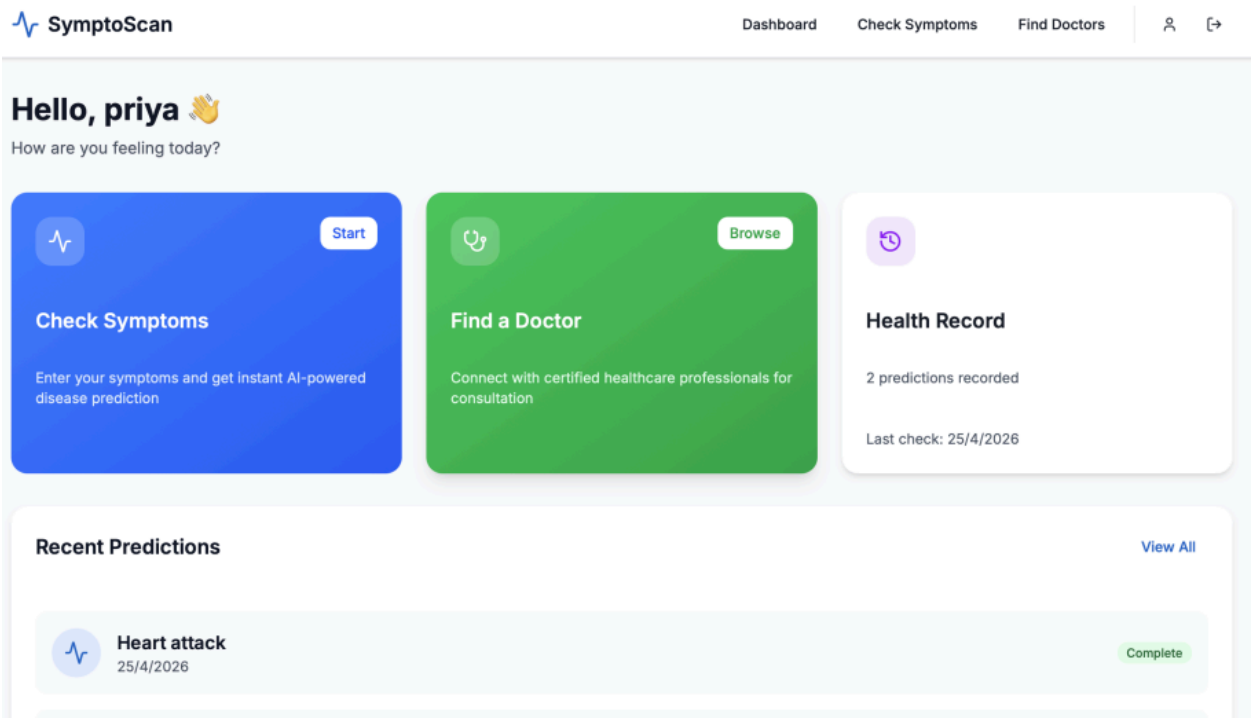

Figure 4.5 — Patient Dashboard

4.1.5 Symptom Input & Disease Prediction

This is the core AI feature of SymptoScan. The patient selects symptoms from a list or types them manually. The frontend sends the symptom list to POST /api/prediction/predict/ where a trained scikit-learn Random Forest model (stored as a pickle file) returns the most probable disease. The result is rendered on the Prediction Result page with the disease name and confidence score.

Test Case ID	Test Scenario	Steps Performed	Expected Result	Status
TC_PRED_01	Disease predicted from selected symptoms	Navigate to /patient/symptoms, select fever + cough + headache, click Predict Disease button	Redirected to /patient/prediction, ML model returns predicted disease name and confidence score, result stored in prediction history via GET /api/prediction/history/	PASSED

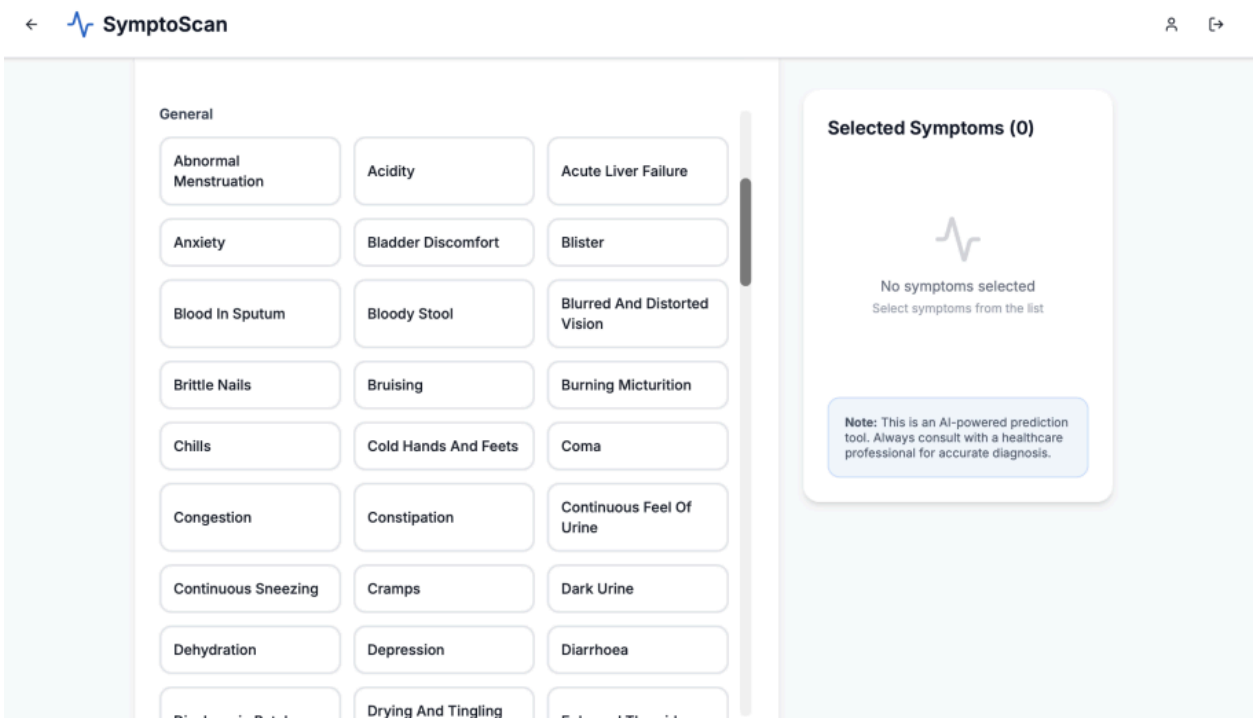

Figure 4.6 — Symptom Input Page (Empty)

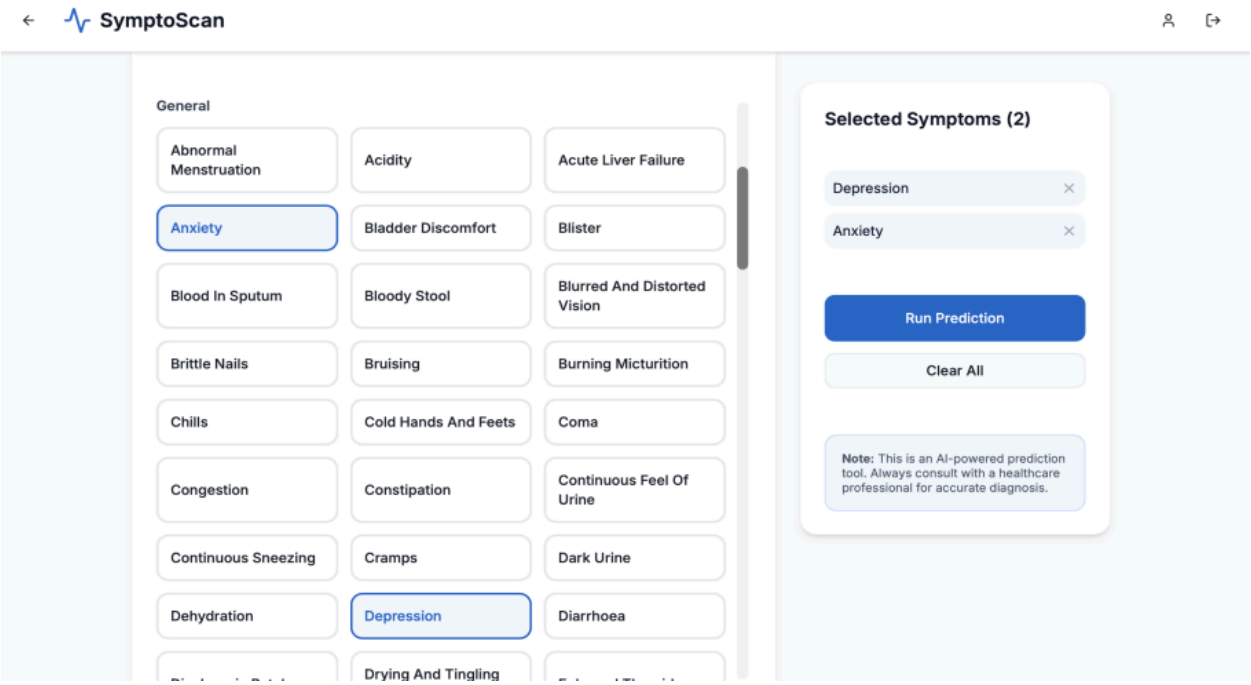

Figure 4.7 — Symptom Input Page (Symptoms Selected)

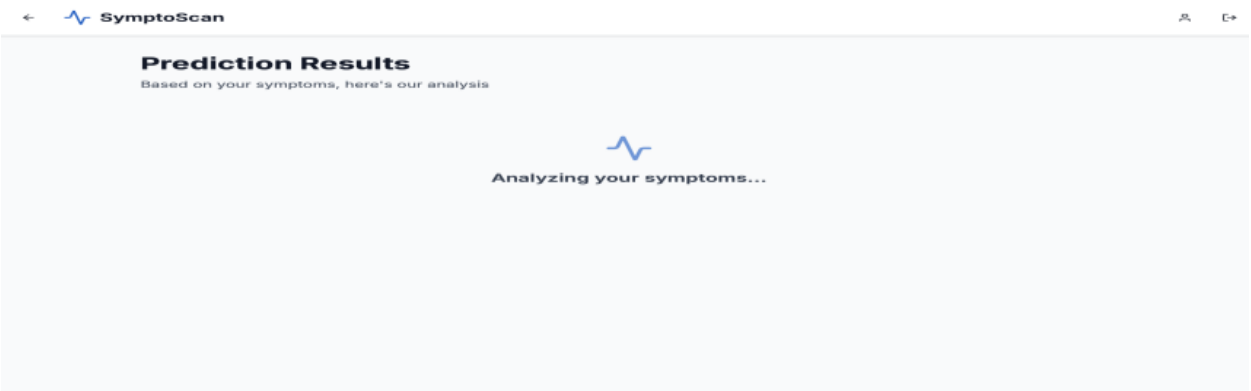

Figure 4.8 — ML Disease Prediction Result

4.1.6 Doctor Listing & Consultation Booking

Patients browse available doctors fetched from GET /api/users/doctors/. Each card shows the doctor's name, specialization, and consultation fee. Clicking Book Consultation opens a Razorpay payment modal. After successful payment, POST /api/consultation/verify/ confirms the transaction and unlocks the chat with that doctor.

Test Case ID	Test Scenario	Steps Performed	Expected Result	Status
TC_PAY_01	Consultation booked after successful payment	Navigate to /patient/doctors, click Book Consultation on any doctor, complete Razorpay test payment using card 4111 1111 1111 1111	Payment verified via POST /api/consultation/verify/, consultation created in database, chat with doctor unlocked at /patient/chat	PASSED

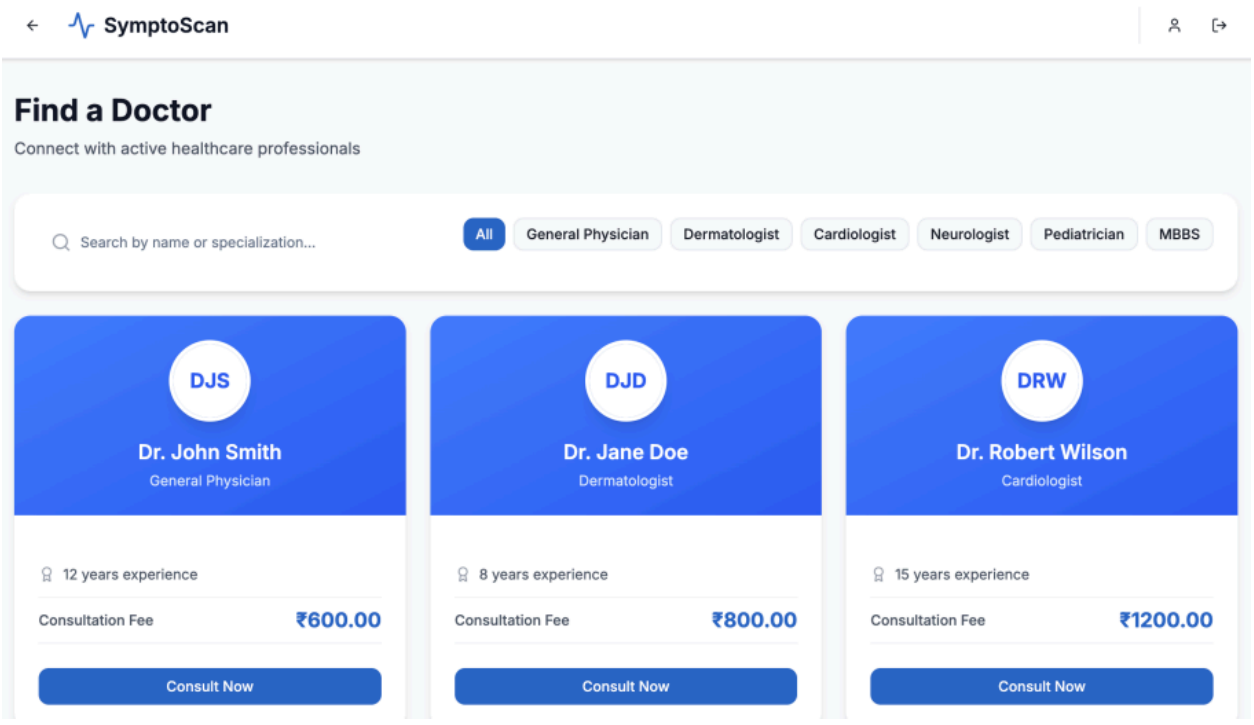

Figure 4.9 — Doctor Listing Page

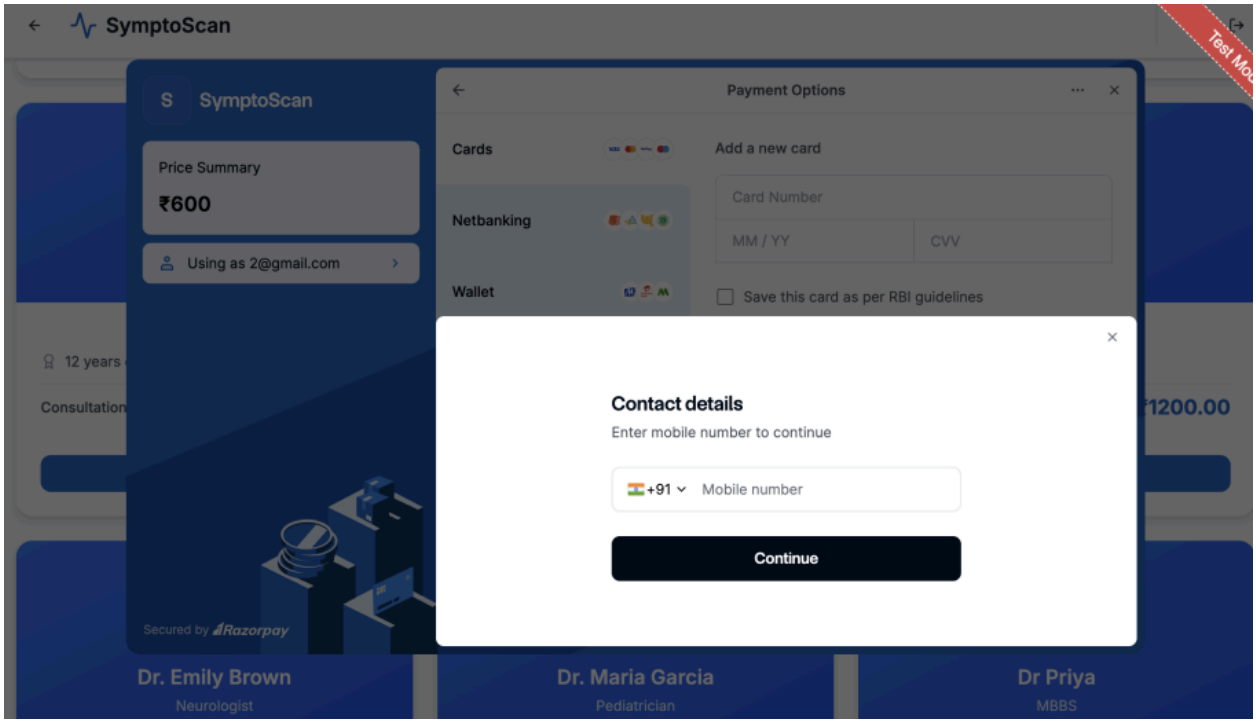

Figure 4.10 — Razorpay Payment Model

4.1.7 Chat Consultation

After a paid consultation is created, the patient and doctor can communicate in real time through the chat interface. Messages are sent via POST /api/consultation/chat/send/ and retrieved via GET /api/consultation/chat/<consultation_id>/. Chat history persists across page refreshes.

Test Case ID	Test Scenario	Steps Performed	Expected Result	Status
TC_CHAT_01	Patient sends message to doctor	Navigate to /patient/chat, type 'Hello Doctor, I need help' in the message box, click Send	Message appears as a sent bubble in the chat window, stored via POST /api/consultation/chat/send/, visible on doctor's chat view	PASSED

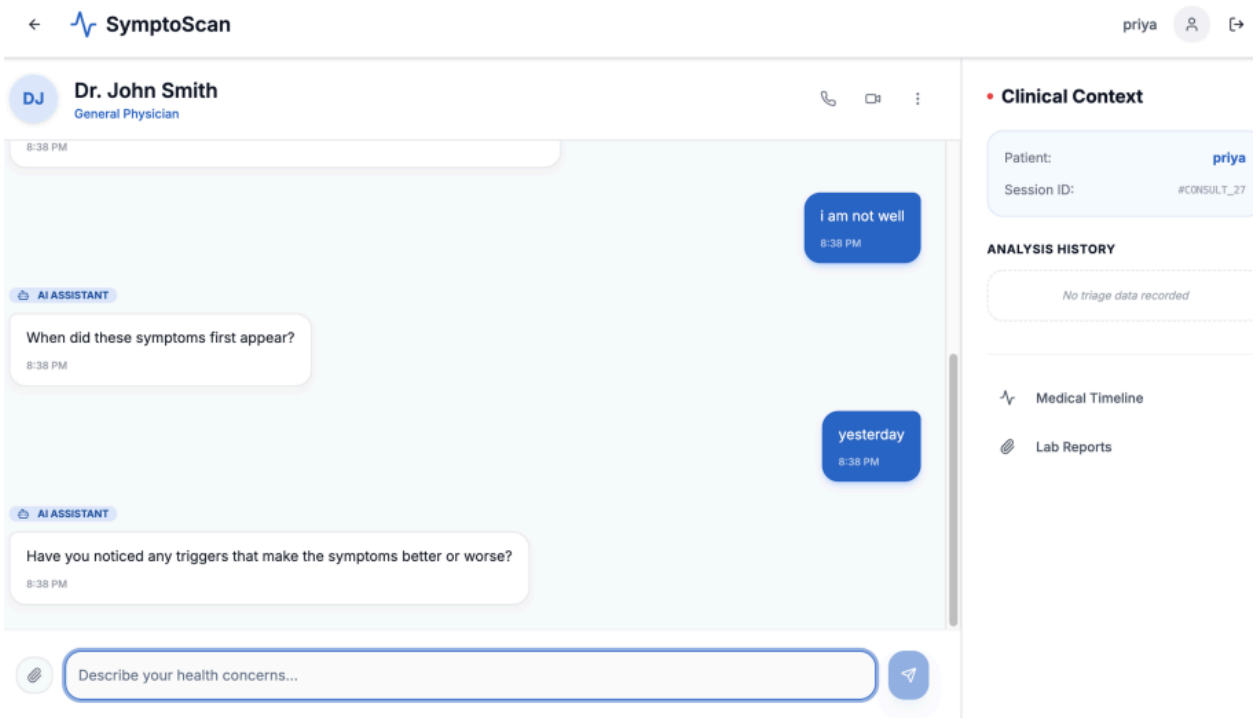

Figure 4.11 — Chat Consultation Window

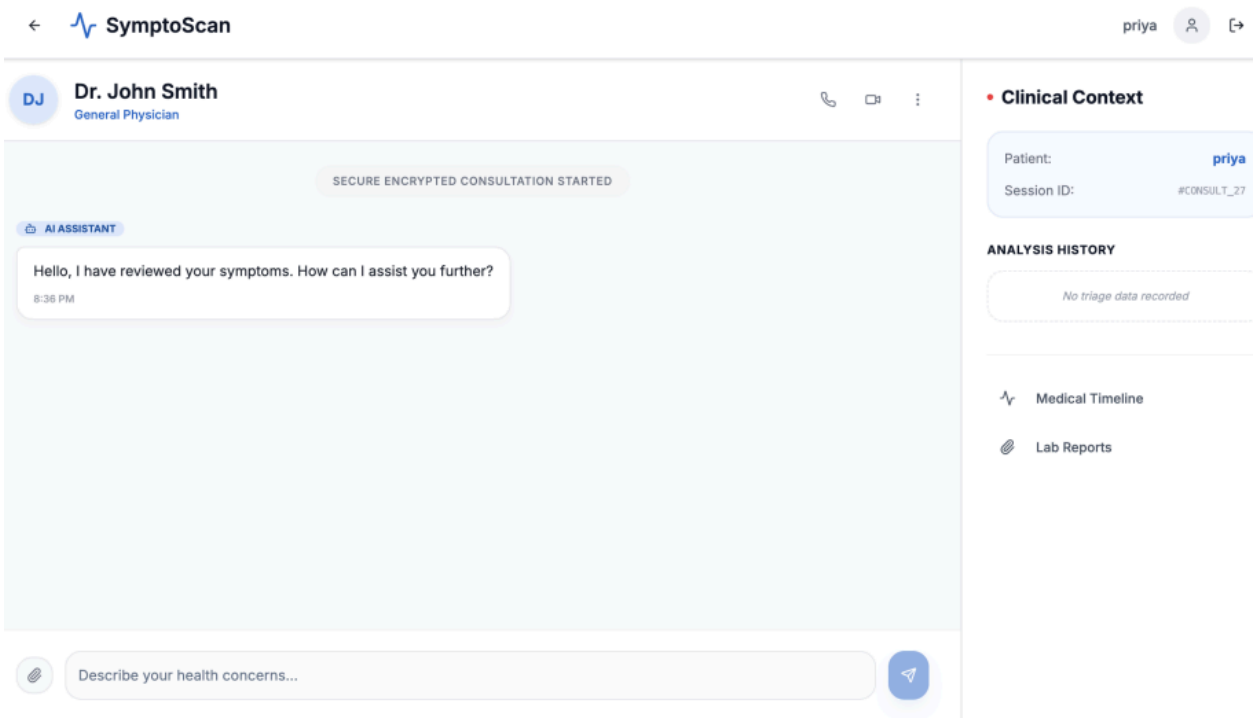

Figure 4.12 — Message Sent to Doctor

4.1.8 Admin Dashboard

The admin dashboard is restricted to users with the admin role (IsAdminUser Django permission). It provides platform-wide statistics including total patients, doctors, predictions, consultations, and revenue fetched from GET /api/users/admin-dashboard/. Admins can also view and manage all registered users.

Test Case ID	Test Scenario	Steps Performed	Expected Result	Status
TC_ADM_01	Admin views platform statistics	Login as admin@symptoscan.com , navigate to /admin/dashboard	Dashboard displays total_patients, total_doctors, total_predictions, total_consultations, total_revenue fetched from GET /api/users/admin-dashboard/ with HTTP 200	PASSED

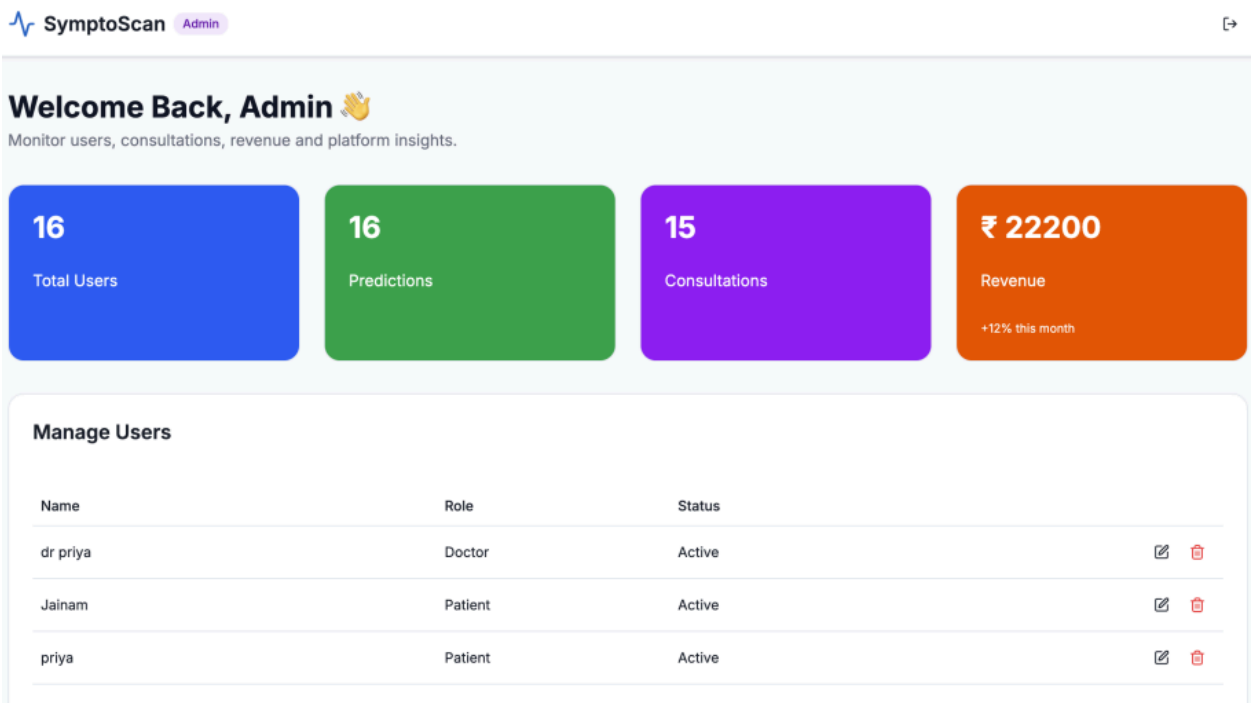

Figure 4.13 — Admin Dashboard Statistics

Figure 4.14 — Payment History Table

4.2 Validation Evidence Summary

The following table consolidates the single validation test case for each module. Every module was manually tested against the live deployment at https://symptoscan-frontend-yjxv.onrender.com and confirmed working as expected.

Module	Test Case ID	Validation Performed	Result
Landing Page	TC_LAND_01	Page loads with hero + navbar visible	PASSED
User Login	TC_AUTH_01	Valid login returns JWT + redirects to dashboard	PASSED
User Registration	TC_REG_01	New account created, HTTP 201 returned	PASSED
Patient Dashboard	TC_DASH_01	All feature cards visible after login	PASSED
Symptom Prediction	TC_PRED_01	ML model returns disease + confidence score	PASSED
Doctor & Payment	TC_PAY_01	Razorpay payment verified, chat unlocked	PASSED
Chat Consultation	TC_CHAT_01	Message sent + stored + visible to doctor	PASSED
Admin Dashboard	TC_ADM_01	Stats returned from API with HTTP 200	PASSED

All 8 module validations passed successfully on April 25, 2026. The SymptoScan platform

demonstrated correct functionality across authentication, AI-based disease prediction, doctor consultation booking, real-time chat, and administrative control.

4.2 Pytest Terminal Output — All 30 Tests Passed

The following screenshot shows the complete pytest terminal output confirming that all 30 test cases passed successfully (30 passed, 1 warning in 97.36s).

4.3 Patient Dashboard — Successful Login

The following screenshot shows the SymptoScan frontend after a successful login with credentials (1@gmail.com / d12345678). The 'Welcome back' greeting is displayed, confirming end-to-end authentication works correctly.

5. Automated Test Source Code

· Source: test_login.py

```
"""
SymptoScan — Login Test Suite
File: test_login.py
8 Test Cases: TC_LOGIN_01 through TC_LOGIN_08
"""

import time
import pytest
import requests

BASE_URL = "http://127.0.0.1:8000"
LOGIN_URL = f"{BASE_URL}/api/auth/jwt/create/"
REGISTER_URL = f"{BASE_URL}/api/auth/users/"

VALID_EMAIL = "1@gmail.com"
VALID_PASSWORD = "d12345678"
WRONG_PASSWORD = "123456578"
ADMIN_EMAIL = "admin@symptoscan.com"
ADMIN_PASSWORD = "admin123"


class TestLogin:

    def test_TC_LOGIN_01_valid_credentials(self):
        """TC_LOGIN_01: Valid credentials → 200 + access + refresh tokens"""
        resp = requests.post(LOGIN_URL, json={
            "email": VALID_EMAIL,
            "password": VALID_PASSWORD
        })
        assert resp.status_code == 200, f"Expected 200, got {resp.status_code}: {resp.text}"
        data = resp.json()
        assert "access" in data, "Response missing 'access' token"
        assert "refresh" in data, "Response missing 'refresh' token"
        assert len(data["access"]) > 20, "Access token appears too short"
```

```
def test_TC_LOGIN_02_wrong_password(self):
    """TC_LOGIN_02: Wrong password → 401"""
    resp = requests.post(LOGIN_URL, json={
        "email": VALID_EMAIL,
        "password": WRONG_PASSWORD
    })
    assert resp.status_code == 401, f"Expected 401, got {resp.status_code}"

def test_TC_LOGIN_03_empty_credentials(self):
    """TC_LOGIN_03: Empty email & password → 400 or 401"""
    resp = requests.post(LOGIN_URL, json={"email": "", "password": ""})
    assert resp.status_code in [400, 401], f"Expected 400 or 401, got {resp.status_code}"

def test_TC_LOGIN_04_unregistered_email(self):
    """TC_LOGIN_04: Unregistered email → 401"""
    resp = requests.post(LOGIN_URL, json={
        "email": "notregistered_xyz@example.com",
        "password": "somepassword123"
    })
    assert resp.status_code == 401, f"Expected 401, got {resp.status_code}"

def test_TC_LOGIN_05_admin_login(self):
    """TC_LOGIN_05: Admin login → 200 + access token"""
    resp = requests.post(LOGIN_URL, json={
        "email": ADMIN_EMAIL,
        "password": ADMIN_PASSWORD
    })
    assert resp.status_code == 200, f"Expected 200, got {resp.status_code}: {resp.text}"
    data = resp.json()
    assert "access" in data, "Admin response missing 'access' token"

def test_TC_LOGIN_06_whitespace_email(self):
    """TC_LOGIN_06: Whitespace in email → not 500"""
    resp = requests.post(LOGIN_URL, json={
        "email": "   ",
        "password": "anything123"
    })
    assert resp.status_code != 500, f"Got unexpected 500 Internal Server Error"

def test_TC_LOGIN_07_register_endpoint_exists(self):
    """TC_LOGIN_07: Register endpoint exists → 400 on empty payload"""
    resp = requests.post(REGISTER_URL, json={})
    assert resp.status_code in [400, 401], (
        f"Expected 400 (validation error) or 401, got {resp.status_code}"
    )

def test_TC_LOGIN_08_response_time_under_10s(self):
    """TC_LOGIN_08: Login response time < 10 seconds"""
    start = time.time()
    resp = requests.post(LOGIN_URL, json={
        "email": VALID_EMAIL,
        "password": VALID_PASSWORD
    })
    elapsed = time.time() - start
    assert elapsed < 10, f"Login took {elapsed:.2f}s, expected < 10s"
    assert resp.status_code == 200
```

· Source: test_integration.py

```
"""
SymptoScan — Integration Test Suite
File: test_integration.py
22 Test Cases across 7 modules
"""

import time
import pytest
import requests

BASE_URL = "http://127.0.0.1:8000"
LOGIN_URL = f"{BASE_URL}/api/auth/jwt/create/"
REFRESH_URL = f"{BASE_URL}/api/auth/jwt/refresh/"
REGISTER_URL = f"{BASE_URL}/api/auth/users/"

VALID_EMAIL = "1@gmail.com"
VALID_PASSWORD = "d12345678"
ADMIN_EMAIL = "admin@symptoscan.com"
ADMIN_PASSWORD = "admin123"


# ─────────────────────────────────────────────────────────────
# Fixtures
# ─────────────────────────────────────────────────────────────

@pytest.fixture(scope="module")
def user_tokens():
    """Acquire JWT tokens for the regular user once per module."""
    resp = requests.post(LOGIN_URL, json={"email": VALID_EMAIL, "password": VALID_PASSWORD})
    assert resp.status_code == 200, f"User login failed: {resp.text}"
    return resp.json()

@pytest.fixture(scope="module")
def admin_tokens():
    """Acquire JWT tokens for the admin user once per module."""
    resp = requests.post(LOGIN_URL, json={"email": ADMIN_EMAIL, "password": ADMIN_PASSWORD})
    assert resp.status_code == 200, f"Admin login failed: {resp.text}"
    return resp.json()

@pytest.fixture(scope="module")
def user_auth_headers(user_tokens):
    return {"Authorization": f"Bearer {user_tokens['access']}"}

@pytest.fixture(scope="module")
def admin_auth_headers(admin_tokens):
    return {"Authorization": f"Bearer {admin_tokens['access']}"}


# ─────────────────────────────────────────────────────────────
# MODULE 1: System Foundations
# ─────────────────────────────────────────────────────────────

class TestSystemFoundations:

    def test_INT_01_backend_reachable(self):
        """INT_01: Backend reachable (not 5xx)"""
        resp = requests.get(f"{BASE_URL}/api/auth/users/me/",
                            headers={"Authorization": "Bearer invalid"})
        assert resp.status_code < 500, f"Got unexpected 5xx: {resp.status_code}"

    def test_INT_02_jwt_login_endpoint_up(self):
```

```
        """INT_02: JWT login endpoint returns expected response"""
        resp = requests.post(LOGIN_URL, json={"email": VALID_EMAIL, "password": VALID_PASSWORD})
        assert resp.status_code == 200
        assert "access" in resp.json()

    def test_INT_03_register_endpoint_up(self):
        """INT_03: Djoser /api/auth/users/ endpoint up"""
        resp = requests.post(REGISTER_URL, json={})
        # Empty payload should return 400 (validation) not 404 or 500
        assert resp.status_code not in [404, 500], f"Endpoint might be down: {resp.status_code}"


# ──────────────────────────────────────────────────────────────
# MODULE 2: Authentication
# ──────────────────────────────────────────────────────────────

class TestAuthentication:

    def test_INT_04_user_token_acquired(self, user_tokens):
        """INT_04: User token acquired successfully"""
        assert "access" in user_tokens
        assert "refresh" in user_tokens

    def test_INT_05_admin_token_acquired(self, admin_tokens):
        """INT_05: Admin token acquired successfully"""
        assert "access" in admin_tokens

    def test_INT_06_wrong_password_returns_401(self):
        """INT_06: Wrong password → 401"""
        resp = requests.post(LOGIN_URL, json={"email": VALID_EMAIL, "password": "wrongpass999"})
        assert resp.status_code == 401

    def test_INT_07_protected_route_without_token(self):
        """INT_07: Protected route without token → 401/403"""
        resp = requests.get(f"{BASE_URL}/api/users/doctors/")
        assert resp.status_code in [401, 403], f"Expected 401/403, got {resp.status_code}"

    def test_INT_08_jwt_refresh_works(self, user_tokens):
        """INT_08: JWT refresh token works → 200 + new access token"""
        resp = requests.post(REFRESH_URL, json={"refresh": user_tokens["refresh"]})
        assert resp.status_code == 200, f"Refresh failed: {resp.text}"
        assert "access" in resp.json()


# ──────────────────────────────────────────────────────────────
# MODULE 3: Users & Doctors
# ──────────────────────────────────────────────────────────────

class TestUsersAndDoctors:

    def test_INT_09_authenticated_user_gets_doctors_list(self, user_auth_headers):
        """INT_09: Authenticated user gets doctors list → 200 + list"""
        resp = requests.get(f"{BASE_URL}/api/users/doctors/", headers=user_auth_headers)
        assert resp.status_code == 200, f"Expected 200, got {resp.status_code}: {resp.text}"
        data = resp.json()
        # Accept both paginated (results key) and plain list
        doctors = data.get("results", data) if isinstance(data, dict) else data
        assert isinstance(doctors, list), "Doctors response should be a list"

    def test_INT_10_doctor_availability_endpoint(self, user_auth_headers):
        """INT_10: Doctor availability endpoint → 200 or 403"""
```

```
        resp = requests.get(f"{BASE_URL}/api/users/availability/", headers=user_auth_headers)
        assert resp.status_code in [200, 403], f"Expected 200/403, got {resp.status_code}"

    def test_INT_11_admin_dashboard_blocked_for_regular_user(self, user_auth_headers):
        """INT_11: Admin dashboard blocked for regular user → 401/403"""
        resp = requests.get(f"{BASE_URL}/api/users/admin-dashboard/", headers=user_auth_headers)
        assert resp.status_code in [401, 403], f"Expected 401/403, got {resp.status_code}"


# ═══════════════════════════════════════════════════════════════
# MODULE 4: Prediction / ML
# ═══════════════════════════════════════════════════════════════

class TestPredictionML:

    def test_INT_12_predict_endpoint_reachable(self, user_auth_headers):
        """INT_12: Predict endpoint reachable when authenticated → 200/400"""
        resp = requests.post(
            f"{BASE_URL}/api/prediction/predict/",
            json={"symptoms": ["fever", "cough", "headache"]},
            headers=user_auth_headers
        )
        assert resp.status_code in [200, 400], f"Unexpected status: {resp.status_code}: {resp.text}"

    def test_INT_13_prediction_history(self, user_auth_headers):
        """INT_13: Prediction history → 200"""
        resp = requests.get(f"{BASE_URL}/api/prediction/history/", headers=user_auth_headers)
        assert resp.status_code == 200, f"Expected 200, got {resp.status_code}: {resp.text}"

    def test_INT_14_prediction_blocked_without_auth(self):
        """INT_14: Prediction blocked without auth → 401/403"""
        resp = requests.post(
            f"{BASE_URL}/api/prediction/predict/",
            json={"symptoms": ["fever"]}
        )
        assert resp.status_code in [401, 403], f"Expected 401/403, got {resp.status_code}"


# ═══════════════════════════════════════════════════════════════
# MODULE 5: Consultation
# ═══════════════════════════════════════════════════════════════

class TestConsultation:

    def test_INT_15_consultation_history(self, user_auth_headers):
        """INT_15: Consultation history → 200/404"""
        resp = requests.get(f"{BASE_URL}/api/consultation/history/", headers=user_auth_headers)
        assert resp.status_code in [200, 404], f"Expected 200/404, got {resp.status_code}"

    def test_INT_16_payment_history(self, admin_auth_headers):
        """INT_16: Payment history (admin) → 200/404"""
        resp = requests.get(f"{BASE_URL}/api/consultation/payments/", headers=admin_auth_headers)
        assert resp.status_code in [200, 404], f"Expected 200/404, got {resp.status_code}"

    def test_INT_17_consultation_blocked_without_token(self):
        """INT_17: Consultation blocked without token → 401/403"""
        resp = requests.get(f"{BASE_URL}/api/consultation/history/")
        assert resp.status_code in [401, 403], f"Expected 401/403, got {resp.status_code}"


# ═══════════════════════════════════════════════════════════════
```

```
# MODULE 6: Admin Dashboard
# ──────────────────────────────────────────────────────────────────────

class TestAdminDashboard:

    def test_INT_18_admin_gets_dashboard_stats(self, admin_auth_headers):
        """INT_18: Admin gets dashboard stats → 200 with total_patients etc."""
        resp = requests.get(f"{BASE_URL}/api/users/admin-dashboard/", headers=admin_auth_headers)
        assert resp.status_code == 200, f"Expected 200, got {resp.status_code}: {resp.text}"
        data = resp.json()
        # At least one expected metric key should be present
        expected_keys = ["total_patients", "total_doctors", "total_users", "total_consultations"]
        found = [k for k in expected_keys if k in data]
        assert len(found) > 0, f"No expected stat keys found in response: {list(data.keys())}"

    def test_INT_19_admin_gets_all_users(self, admin_auth_headers):
        """INT_19: Admin gets all users → 200"""
        resp = requests.get(f"{BASE_URL}/api/users/all/", headers=admin_auth_headers)
        assert resp.status_code == 200, f"Expected 200, got {resp.status_code}: {resp.text}"

    def test_INT_20_manage_nonexistent_user(self, admin_auth_headers):
        """INT_20: Manage non-existent user → 404"""
        resp = requests.get(f"{BASE_URL}/api/users/manage/999999/", headers=admin_auth_headers)
        assert resp.status_code == 404, f"Expected 404 for non-existent user, got {resp.status_code}"


# ──────────────────────────────────────────────────────────────────────
# MODULE 7: Performance
# ──────────────────────────────────────────────────────────────────────

class TestPerformance:

    def test_INT_21_login_under_10_seconds(self):
        """INT_21: Login under 10 seconds"""
        start = time.time()
        resp = requests.post(LOGIN_URL, json={"email": VALID_EMAIL, "password": VALID_PASSWORD})
        elapsed = time.time() - start
        assert elapsed < 10, f"Login took {elapsed:.2f}s — exceeds 10s limit"
        assert resp.status_code == 200

    def test_INT_22_doctors_list_under_10_seconds(self, user_auth_headers):
        """INT_22: Doctors list under 10 seconds"""
        start = time.time()
        resp = requests.get(f"{BASE_URL}/api/users/doctors/", headers=user_auth_headers)
        elapsed = time.time() - start
        assert elapsed < 10, f"Doctors list took {elapsed:.2f}s — exceeds 10s limit"
        assert resp.status_code == 200
```